Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712000222, Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026-27.	Semester:	1st
Name of the Student:	Ch. pradeep	Roll Number:	A24126552434
Name of the Lab:	FULL STACK.	Timings:	8:50 - 12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva-Voce (2Marks)			
1/7	1	3	3	4	1	12	10	R
8/7	2	3	3	4	1	12	10	R
15/7	3	3	3	4	2	13	10	R
22/7	4	3	3	4	2	13	10	R
29/7	5	3	3	4	1	12	10	R
5/8	6	3	3	4	1	12	10	R
9/8	7	3	3	4	2	13	10	R
16/9	8	3	3	4	1	12	10	R
23/9	9.	3	3	4	1	12	10	R
30/9	10	3	3	4	2	13	10	R

Name of the Faculty:

Faculty Signature: R

INDEX

ANITS

S.No	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
1.	6/7/26	Introduction to HTML & CSS	CO1	12	
2.	15/7/26	static webpage using HTML & CSS	CO1	12	
3.	22/7/26	HTML codes using audio & redio Tags.	CO1	13	
4.	29/7/26	program ~~webpage~~ using java Script	CO1	13	
5.	29/7/26	Dynamic webpage using javascript	CO1	12	
6.	5/8/26	Implement modules in Nodejs.	CO2	12	
7.	12/8/26	Develop mini application using Express & Nodejs	CO2	13	
8	19/8	code Tantra ~~per~~ (Form).	CO2	12	
9.	2/9	code Tantra (portfolio).	CO2	12	
10	9/9	code Tantra (simple calculator).	CO2	13	
11	16/9	create a functional & class.	CO3	12	
12	23/9	Implement HTTP method.	CO3.	13	

Introduction to HTML and CSS

→ HTML [Hyper text markup Language] is the standard markup language used to create and structure content on web pages. It defines the basic structure of a webpages using elements and tags such as headings, paragraph, links, images, Lists, Tables, Forms and other multimedia content. HTML documents are interpreted by web browser to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a web page.

⇒ CSS (cascading style sheets) is a stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, spacing, borders, alignment and layouts. CSS also supports responsive design, allowing web pages to adapt to different screen size and devices. CSS can be applied using inline, internal or external stylesheet.

⇒ HTML and CSS work together to develop webpages, where HTML provides the structure and content, while CSS provides the styling layout, visual presentation. The combination of both forms the foundation of modern web development and enables the creation of organized attractive, and user-friendly web interface.

Difference between MERN stack and Full stack

→ MERN stack is a Javascript based technology stack used for developing modern web applications. MERN stands for mongoDB, Express.js, React.js, Node.js mongoDB is used for database management, Express.js handles the backend framework and Api's, React.js used to develop fronted user interface, and Node.js provides the server side runtime environment.

→ Full-stack development refers to the development of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used for designing the user interface, Implementing server-side logic managing database.

HTML components

1. HTML Tags - Define the structure and elements of a web page.
2. Headings - used to display titles and headings using <h1> to <h6>
3. paragraphs - used to display text using the <p> tag.
4. Links - used to Navigate between webpages using <a> tag.
5. Images - used to display image using <image> tag.
6. Lists - used to create ordered and unordered lists using <OI>, <UI>,
7. Tables - used to organize data in rows, and columns.
8. Forms - used to collect user input using elements such as text boxes, button, and checkboxes.
9. Div and span - used to group and organize webpage content.
10. Semantic Elements - elements such as <header> <nav>, <selection>, <article>, <footer>.

CSS components

1. Selectors – Select HTML elements to apply styles.
2. properties – Define the visual characteristic of elements.
3. values – specify the setting of a CSS property.
4. colors – Define text, background, border colors.
5. Fonts – control font family, size style and weight.
6. Backgrounds – Set background colors and images.
7. Borders – Define the border style, width and radius.
8. Margin – controls the space outside an elements.
9. padding – controls the space inside element.
10. Layout – controls the arrangement of elements using Flexbox, Grid and positioning.

HTML Vs HTML5

HTML	HTML5.
Older version of HTML	Latest major version of HTML
Limited multimedia support	supports audio and video.
uses External plugins for multimedia	No External plugs needed.
Has fewer semantic tags	provides semantic tags.
NO <canvas> element	support <canvas>
Limited web storage	support local storage
complex DOCTYPE declaration	simple <!DOCTYPE HTML>

CSS Vs CSS3

CSS	CSS3.
Earlier version of CSS	Advanced version of CSS
Basic styling features	provides advanced styling features.
Limited animations	supports animations and transitions.
NO flexible layout systems	supports flexbox and Grid.
Limited visual effects	supports shadows, gradients, effects.
NO media queries	supports media queries.

design a static web page using HTML and CSS.

pomodoro timer, using HTML, CSS.

```
<!DOCTYPE html>
<html>
<head>
    <title pomodoro timer</title>
    <style>
        body {
            font-family : Arrial;
            background  : #f5f5f5;
            text-align  : center;
            padding     : 50px;
        }

        .timer {
            width : 300px;
            margin : auto;
            padding : 30px;
            background : white;
            border-radius : 15px;
            box-shadow : 0 0 10px gray;
        }

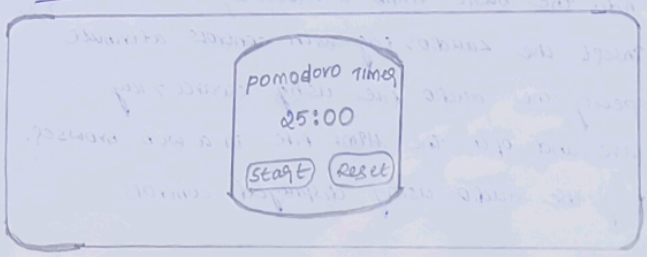
        h1 {
            color : #e74c3c;
        }

        .time {
            font-size : 50px;
            margin    : 20px;
        }

        button {
            padding : 10px 20px;
            margin  : 5px;
            border  : none;
            border-radius : 5px;
            background : #e74c3c;
            color : white;
        }

    </style>
</head>
<body>
    <div class = 'timer'>
        <h1> pomodoro timer </h1>
        <div class = 'time'> 25:00 </div>
        <button> start </button>
        <button> reset </button>
    </div>
</body>
</html>
```

output:



i) Design an HTML code using audio tags.

code:-

```
<!DOCTYPE html>
<html>
<head>
        <title> Audio Example </title>
</head>
<body>
        <h2> My favorite song </h2>
        <audio controls>
            <source src = "song.mp3" Type = "audio/mp3">
            your browser does not support the audio tag
        </audio>
</body>
</html>
```

O/p:- Displays an audio player with play, pause and volume controls.

steps:-
create a new HTML file
Add the basic HTML structure
Insert the <audio> tag with controls attribute
Specify the audio file using <source> Tag
Save and open the HTML file in a web browser.
play the audio using displayed controls.

ii) Design a HTML code using video tags.

code:

```
<!DOCTYPE html>
<html>
<head>
        <title> video Example </title>
</head>
<body>
        <h2> my video </h2>
        <vedio width = '400' controls>
            <source src = 'vedio.mp4" type = "vedio/mp4">
        </vedio>
</body>
</html>
```

O/p:- Display a vedio player with play, pause, volume, and full screen controls.

steps:-
create a new HTML file
Add a basic HTML structure
Add a heading using the <h2> tag
Insert the <video> tag with controls attribute
Specify the video file using <source> tag
Save the file and open it in web browser
play the vedio using displayed controls.

Write a program using Javascript (arrays, functions)

code:-

```
let numbers = [12,45,8,23,56,19,34,67,90,11];
function displayArray(arr){
    console.log("Array Elements:");
    for (let i=0; i<arr.length; i++){
     console.log(arr[i]);
    }
}
function findSum(arr){
    let sum=0;
    for (let i=0; i<arr.length; i++){
     sum += arr[i];
    }
    return sum;
}
```

Experiment-04

Aim: Develop dynamic webpage using Java script.

program:-

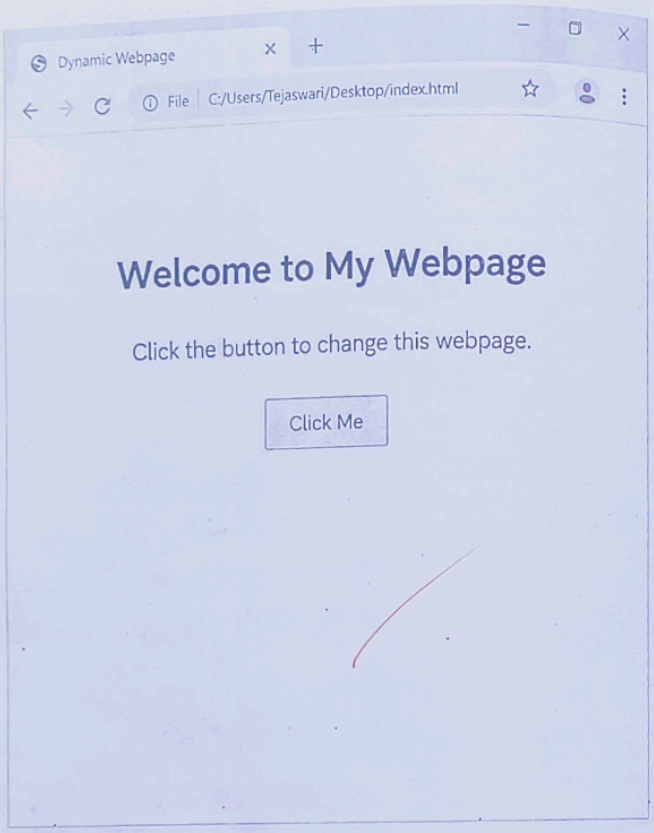
```
<!DOCTYPE html>
<html>
<head>
    <title> Dynamic webpage </title>
    <style>
        body{
            text-align : center;
            font-family : Arial;
            margin-top : 100px;
        }
        button{
            padding : 10px 20px;
            font-size : 16px;
            cursor : pointer;
        }
    </style>
</head>
<body>
    <h1 id = "heading"> welcome to my webpage
    </h1>
    <p id = "message"> click the button to
        change this webpage. </p>
```

<button onclick = "changepage()">Click me</button>

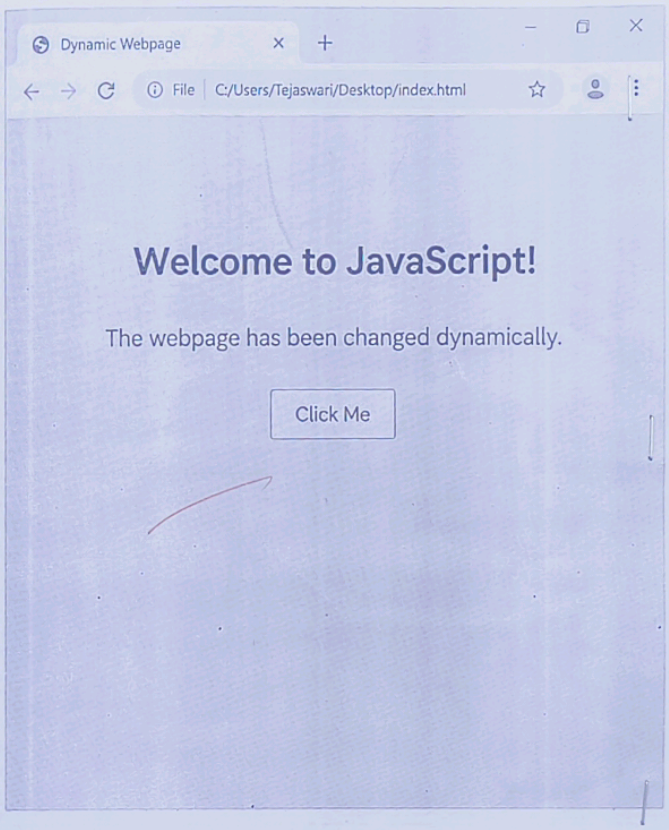
<script>
function changepage(){

After click the button;

welcome to javascript!
The webpage has been dynamicaay changed



Before clicking the button



After clicking the button

Experiment - 05

05 Implement modules in Node.js

In Node.js, you can implement modules using two distinct systems: common js (cjs), which is traditional default and ES module [esm], the modern java script standard

1. The common js method (default):-

common js uses modules exports to share code and require() to import it this system load modules synchronization usily.

a. create the module (math.js)

```
const add = (a,b) => a+b;
const subtract = (a,b) => a-b;
module.exports = {
    add,
    subtract
};
```

b. use the module (app.js)

```
const math = require("./math");
console.log(math.add(10,5));
console.log(math.subtract(10,5));
```

Output:- 15

3.

2. The ES modules method (modern standard).

ES modules use Export to share code and important to use it. To make Node.js recognize ES modules. you must either use the .mjs file extension or add type: 'module' to you package.json file

a. create the module (calculator.js)

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b.
const calculator name = "supercate";
export default calculator name;
```

b. use the module (main.js)

```
import calculator name, {multiply, divide}
        from './calculator.js';
console.log(calculator name);
console.log(multiply(4,2));
console.log(divide(4,2));
```

output:-

Supercate

8

2

Using Built in core modules:-

Node.js also comes equipped with dozens of built in core modules (like fs for file system or path for file routing) that don't need installation we will load them by module Name:

```
// common.js style
const fs = require('fs')
// ES module style.
import fs from 'fs';
```

Here are some of the most commonly ones;

1. fs = file system operation
2. http = http server and client
3. path = file path utilitifies.
4. os = operating system utilities.
5. events = event handling
6. util = utility functions
7. stream = stream handling
8. crypto = cryptographic function
9. url = URI pursing
10. querystring = URI query string handling

Example : simple HTTP server.

```
http.createServer((req, res) => {
  res.writeHead(200, {'content-Type':
    'text/html'});
  res.end('Hello World!');
});
server.listen(8080);
```

Output:- Hello World.

Experiment - 06

Develop mini application using express and Node js

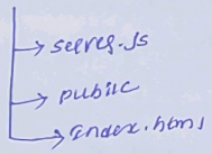
Simple mini application using Node.js and express.js

student management system.

1. Add a student
2. View all student
3. delete a student
4. uses Express js routes
5. Simple HTML interface

- project structure :

student app/



Server.js

```
const express = require("express");
const app = express();
const PORT = 3000;
let students = [];
app.use(express.json());
app.use(express.static("public"));

app.get("/student", (req, res) => {
    res.json(students);
});

// Add
app.post("/student", req, res) => {
    const student = {
        id = student.length + 1,
        name = req.body.name,
        cause : req.body.cource.
    };
    student.push(student);
    res.json({message : 'student added successfully.
        student : student});
});

// Delete
app.delete("/student/:id", (req,res) => {
const id = parseInt(req.params.id);
students = students.filter(student => student.id != id)
res.json({
    message : 'student deleted successfully'});
});

//start server
app.listen(PORT,() => {
    console.log('server running at http://local
        host:${PORT}');
});
```

public/index.html:

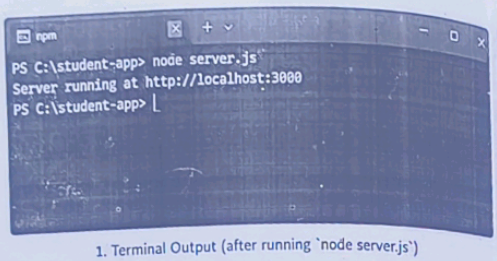
```
<!DOCTYPE html>
<html>
<head>
    <title>student management</title>
</head>
<body>
    <h1>student management</h1>
    <input type="text" id="name" placeholder=
        "enter student name">
    <input type="text" id="cause" placeholder
        ="enter course">
    <button on click="add structure()">Add
        student</button>

    <h2>student list</h2>
    <ul id="student list"></ul>
    <script>
        function add student(){
            let name = document.getElementById('name').
                value;
            let course = document.getElementById('course').
                value;
            fetch('/student',{
                method: "post",
                headers:{
                    "content-type": application/json'},
                body: JSON.stringify({
                    name: name,
                    course: course,
                })
            })
            .then(response => response.json())
            .then(data =>{
                alert(data.message());
            document.getElementById('name').value = "";
            document.getElementById("course").value = "";
                display students();
            });
        }
        function display students(){
            fetch("/students").then(response => response.json())
            .then(student) =>{
                let list = document.getElementById('student list');
                list.innerHTML = "";
                student.forEach(student =>{ let li =
                document.createElement('li')
                    li.innerHTML = student.id + "_" +
                        student.name + "_" +
                        student.course + "<button
            onclick="delete student('"+student.id+"')">delete
                </button>";
                list.appendChild(li);
                });
            });
        }
```

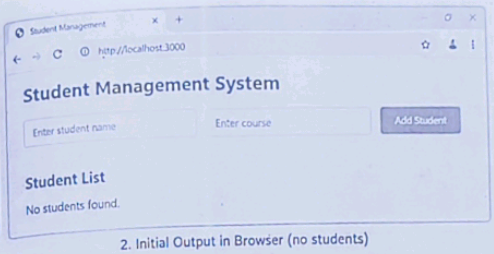
function deleteStudent(id){

fetch('/students/'+id,{method:'DELETE'})

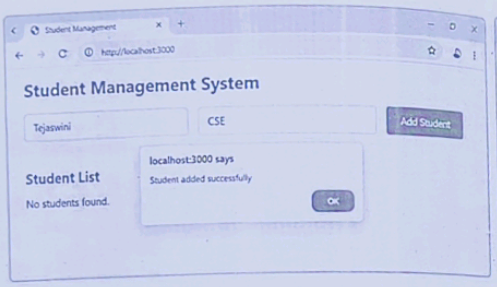
.then(response => response.json())



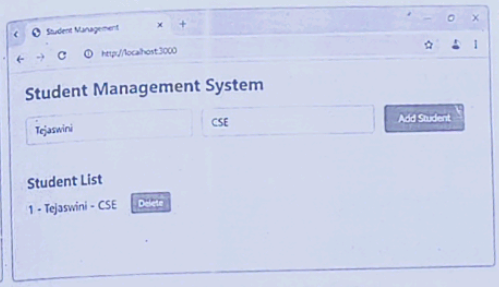
1. Terminal Output (after running `node server.js`)



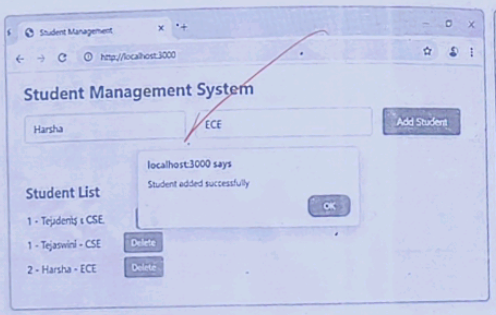
2. Initial Output in Browser (no students)



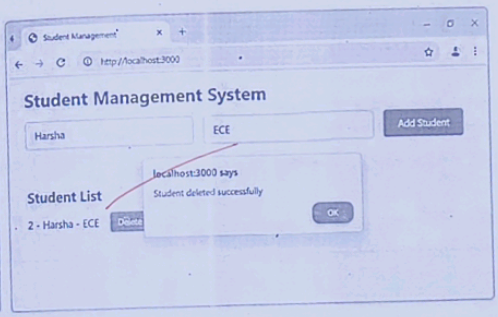
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment-07

1. create a functional components and class components using react.

Function component:-

A function component is a javascript function that returns JSX to display the user interface. It uses React HOOKS such as useState to manage state and update the webpage dynamically

Program:-

```
import React, {useState} from
'react';
function StudentFunctional(){
    const [name, setName] = useState("cherry");
    const [course, setCourse] = useState('CSM');
    const [count, setCount] = useState(0);
    const changeName = () => {
        setName('cherry');
    };
    const changeCourse = () => {
        setCourse("CSD");
    };
```

```
const incrementCount = () => {
    setCount(count+1);
};
const increment
return (
  <div style = {{
    border = '2px solid blue',
    padding = '20px'
    margin = "20px",
    borderRadius = '10px',
    textAlign = 'center'
  }}>
  <h1> Student Details </h1>
  <h2> Functional component </h2>
  <p> <b> Name: </b> {name} </p>
  <p> <b> course : </b> {course} </p>
  <button onClick = {changeName}>
      changeName</button>
  <button onClick = {changeCourse}> change
      course </button>.
  <button onClick = {incrementCount}> Increment
      </button>.
  </div>
);
```

Output:- When the user clicks the button, the name, course and count change dynamically without reloading the webpage

Class component

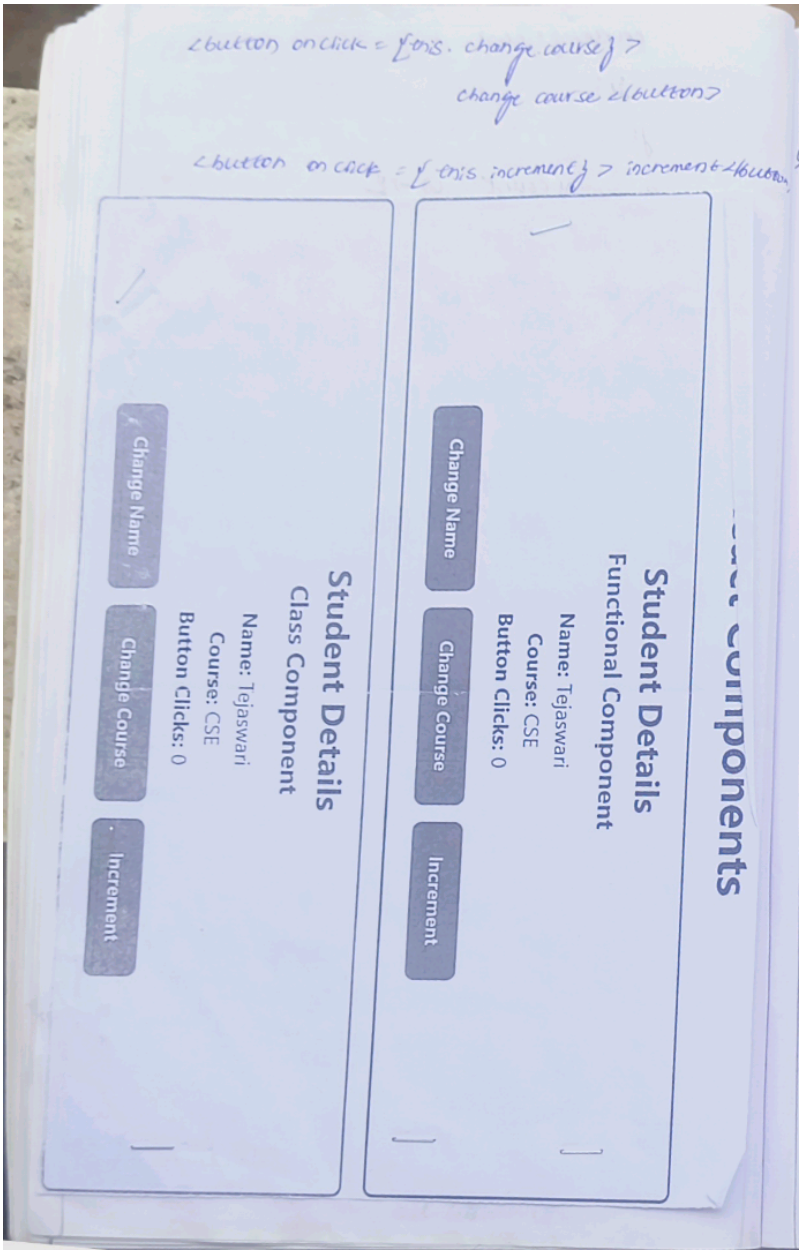
A class component is an ES6 Javascript class that extends React component. It uses the render() method to return JSX and manages state using this.state and this.setState().

Program:-

```
import React, {Component} from
'react';
class StudentClass extends Component{
    constructor(props){
        super(props);
        this.state={
            name: 'cherry',
            course: 'computer science',
            count: 0
        };
    }
    changeName = () =>{
        this.setState({
            name: 'cherry'
        });
    changeCourse = () =>{
        this.setState({
            course: 'CSM'
        });
    };
    incrementCount = () =>{
        this.setState((prevState) =>({
            count: prevState.count+1
        }));
    };
    render(){
        return(
            <div style={{
                border: '2px solid green',
                padding: '20px',
                margin: '20px',
                borderRadius: '10px',
                textAlign: 'center'
            }}>
            <h1>Student Details</h1>
            <h2>Class Component</h2>
            <p><b>Name:</b>{this.state.name}</p>
            <p><b>Course:</b>{this.state.course}</p>
            <p><b>Button clicks:</b>{this.state.count}</p>
            <button onClick={this.changeName}>
                changeName</button>
```

<button onClick={this.changeCourse}>
changeCourse</button>

<button onClick={this.increment}>increment</button>



Experiment-08

Implement Http methods using JSON server

Aim:- To implement HTTP methods GET, POST, PUT, PATCH and Delete using JSON server to perform CRUD (create, Read, Update, and Delete) operation on student records.

Theory:- JSON server is a tool used to create a fake REST API using JSON file. It allows us to perform HTTP operation without developing a backend server from scratch. HTTP methods are used to communicate b/w a client & a server.

- GET → retrieves data from the server
- POST → adds new data to the server
- PUT → replace an existing record
- PATCH → update selected fields of a record
- DELETE → removes the record from the server.

2. create a JSON Server:

Step 1: Initialize the project

open the vs code terminal and execute the following commands

```
MKdir JSONserver
cd JSONserver
npm init -y.
npm install json server.
```

Step 2:- create db.json.

create a file named db.json and add the following student data.

```
{
  "student": [
    {
      "id": "1",
      "name": "cherry",
      "course": "CSM",
      "marks": 90"
    },
    {
      "id": "3"
      "name": "mouni"
      "course": "EEE"
      "marks": 90
    }
```

Step 3:- Start the Json server.

For the current JSON server version, execute:

npx json-server db.json

The server start at:

http://localhost:3000

The student records can be accessed using

http://localhost:3000/students.

Output:-

2. Implementation of HTTP methods:-

a. GET Method:- The GET method is used to retrieve student records from JSON server.

code:-

```
fetch ('http://localhost:3000/students')
  .then (response => response.json()).
  .then (data => {
        console.log (data);
  })
  .catch(error => {
        console.log(error);
  });
```

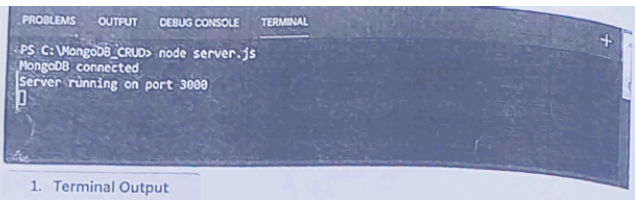
output:-

b. post method:- The post method is used to insert a new student record into the server.

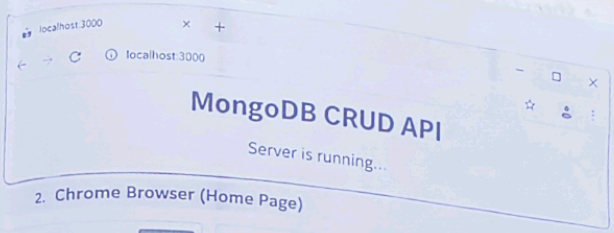
code:-

```
fetch ('http://localhost:3000/student', {
    method : 'post',
    header : {
        'content-Type' : 'application/json'
    },
    body : json.stringify ({
        name : 'cherry',
        course : 'CSE',
        malks : 92
    })
})
  .then (response => response.json())
  .then (data => {
        console.log (data);
  }).
  .catch(error => {
        console.log (error);
  });
```

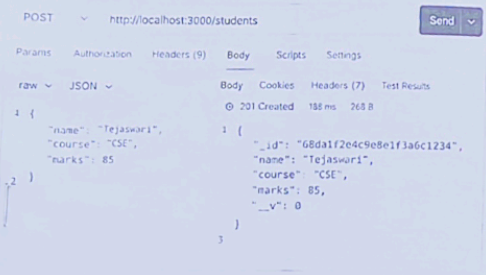
output:-



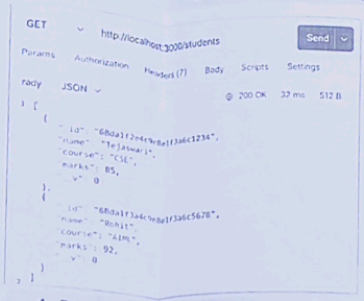
1. Terminal Output



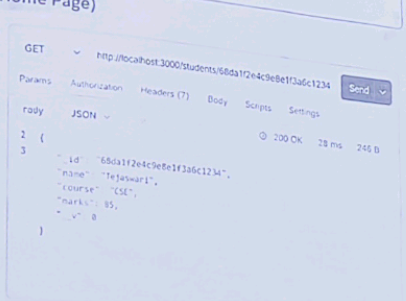
2. Chrome Browser (Home Page)



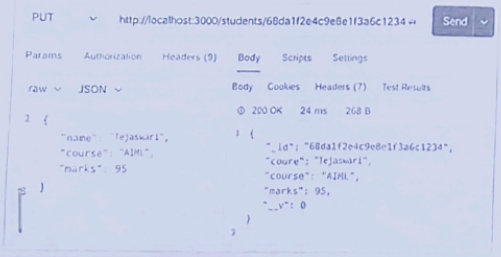
3. Create (POST) – Add Student



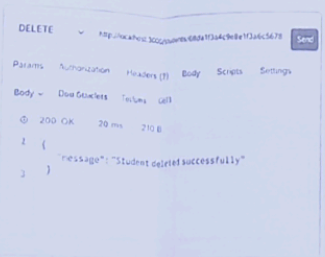
4. Read (GET) – Get All Students



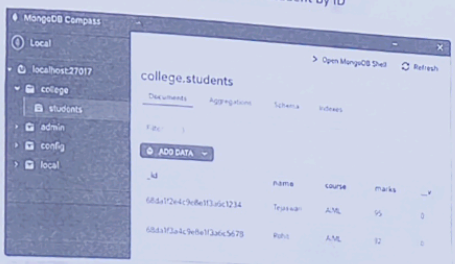
5. Read (GET) – Get Student by ID



6. Update (PUT) – Update Student

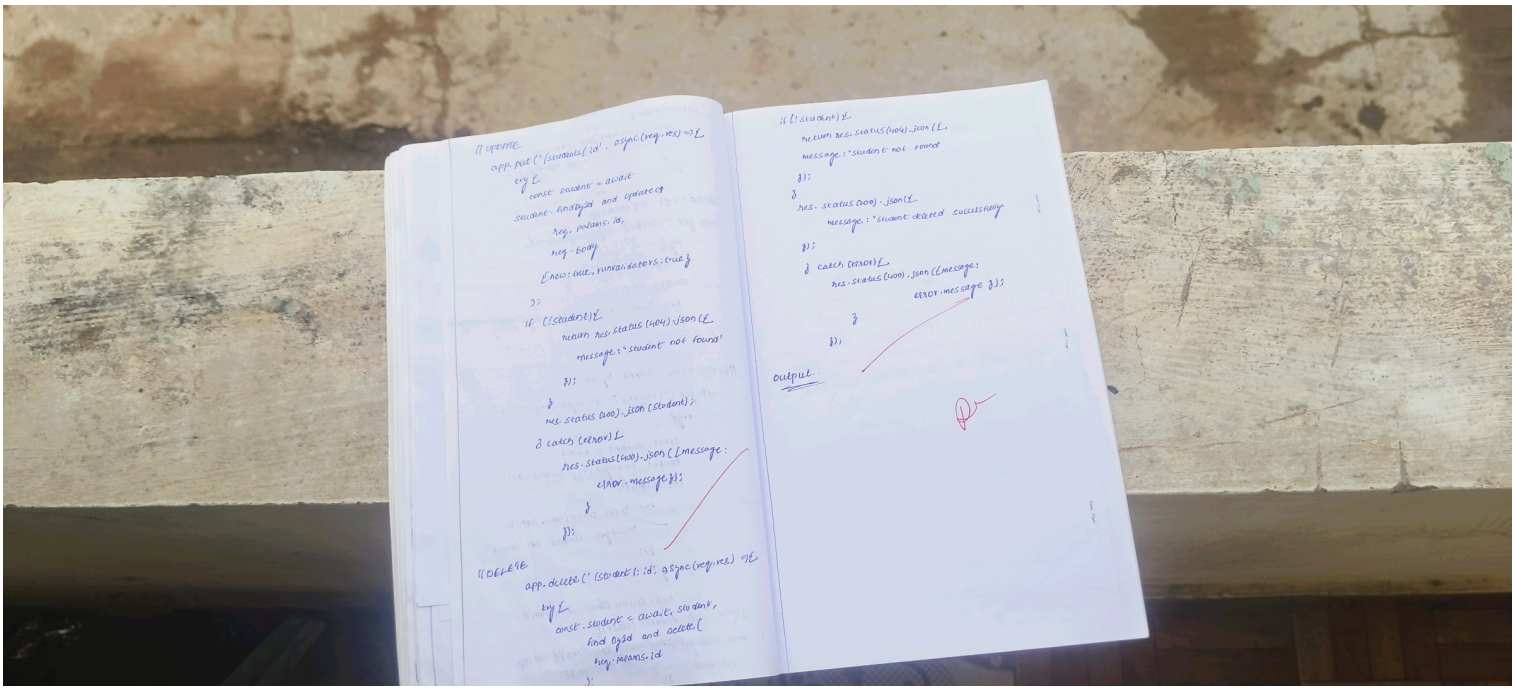


7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View

const PORT = 3000;
app.use(express.json());
mongoose.connect('mongodb://127.0.0.1:27017/college')


// UPDATE
app.put('/students/:id', async (req, res) => {
try {
const student = await
Student.findByIdAndUpdate(
req.params.id,
req.body,
{new: true, runValidators: true}
);
if (!student){
return res.status(404).json({
message: "student not found"
});
}
res.status(200).json(student);
} catch (error){
res.status(400).json({message:
error.message});
}
});
// DELETE
app.delete('/students/:id', async (req,res) =>{
try{
const student = await Student.
findByIdAndDelete(
req.params.id
);
if(!student){
return res.status(404).json({
message: "student not found
});
}
res.status(200).json({
message: "student deleted successfully
});
} catch (error){
res.status(500).json({message:
error.message});
}
});
output